

EC7204: CLOUD COMPUTING

ASSIGNMENT 02 – GROUP PROJECT

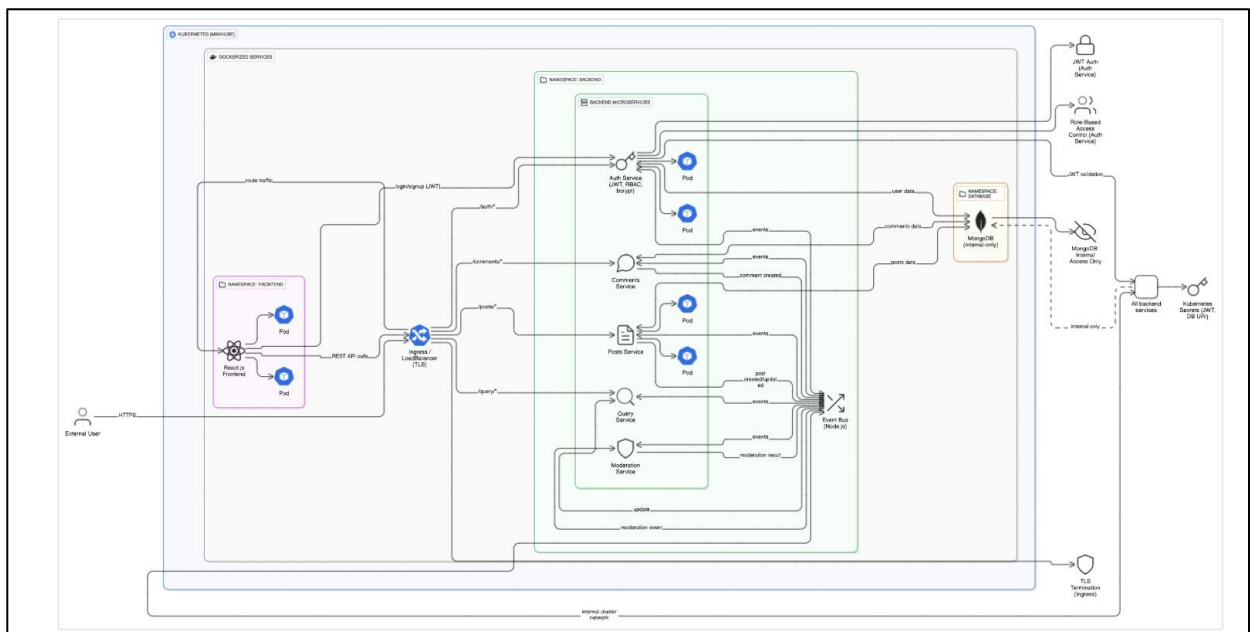
GitHub Link: <https://github.com/Dasundh99/content-filter-microservices-app.git>

REG NO : 3818/ 3964/ 4061/ 4146

1. Introduction

Online platforms produce enormous amounts of user-generated material, such as posts, comments, and multimedia, in today's digital world. One of the biggest challenges that social media platforms, discussion boards, and online communities confront is making sure that this information stays suitable and safe. The scalability, dependability, and security requirements necessary for such systems are difficult for traditional monolithic applications to meet. Our group created and put into operation a cloud-native microservices-based application that filters offensive material from user posts and comments in order to solve this problem. The application exemplifies the fundamentals of cloud computing, such as high availability, scalability, security, synchronous and asynchronous communication, and contemporary deployment techniques. Modular development and horizontal scaling are made possible by the independent implementation of each functionality, including handling postings, managing comments, moderating material, and collecting results. Because Docker and Kubernetes are used in the system's deployment, containerized microservices can operate dependably in a variety of contexts. The system interfaces with MongoDB for data management, guaranteeing scalable and permanent storage. This project demonstrates how cloud-native architectures may efficiently address real-world issues by fusing DevOps principles, container orchestration, and event-driven communication.

2. Architecture



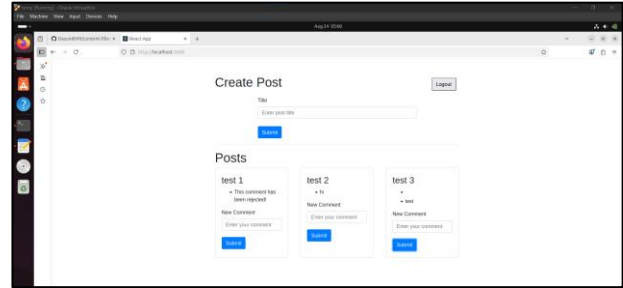
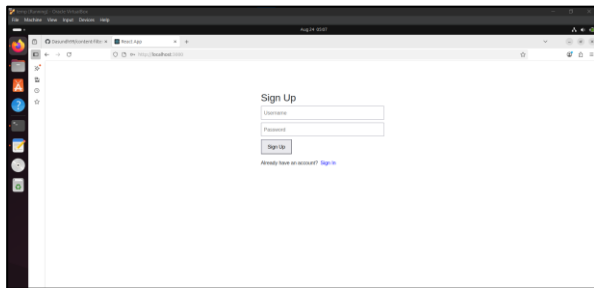
3. Implementation Steps

Services Running

```
temp [running] - Oracle VirtualBox
File Machine View Input Devices Help

vboxuser@temp: ~/Desktop/New Folder/content-filter-microservices-app
vboxuser@temp: ~/Desktop/New Folder/content-filter-microservices-app$ docker ps
CONTAINER ID   IMAGE                                     COMMAND                  NAMES                CREATED      STATUS      PORTS
bc4cd7e93d8c   content-filter-microservices-app-frontend  "docker-entrypoint.s..." client                18 minutes ago Up 18 minutes 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
84dae7b69aaf   content-filter-microservices-app-query-service  "docker-entrypoint.s..." query-service        18 minutes ago Up 18 minutes 0.0.0.0:8003->8003/tcp, [::]:8003->8003/tcp
23983a61a373   content-filter-microservices-app-posts-service  "docker-entrypoint.s..." posts-service        18 minutes ago Up 18 minutes 0.0.0.0:8001->8001/tcp, [::]:8001->8001/tcp
ed74959aefee   content-filter-microservices-app-moderation-service  "docker-entrypoint.s..." moderation-service   18 minutes ago Up 18 minutes 0.0.0.0:8004->8004/tcp, [::]:8004->8004/tcp
dbc92dbb27cc   content-filter-microservices-app-comments-service  "docker-entrypoint.s..." comments-service      18 minutes ago Up 18 minutes 0.0.0.0:8002->8002/tcp, [::]:8002->8002/tcp
f9d56e0718cb   content-filter-microservices-app-event-bus      "docker-entrypoint.s..." event-bus             18 minutes ago Up 18 minutes 0.0.0.0:8005->8005/tcp, [::]:8005->8005/tcp
a27503a792b8   content-filter-microservices-app-auth-service   "docker-entrypoint.s..." auth-service          18 minutes ago Up 18 minutes 0.0.0.0:8006->8006/tcp, [::]:8006->8006/tcp
ac72d7c5a5cb   mongo:4.4                                       "docker-entrypoint.s..." mongo                 18 minutes ago Up 18 minutes 0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp
ca89715de301   gcr.io/k8s-minikube/kicbase:v0.0.47          "/usr/local/bin/entr..." minikube              11 hours ago  Up About an hour  127.0.0.1:32768->22/tcp, 127.0.0.1:32769->2376/tcp, 127.0.0.1:32770->5000/tcp, 127.0.0.1:32771->8443/tcp, 127.0.0.1:32772->32443/tcp
vboxuser@temp: ~/Desktop/New Folder/content-filter-microservices-app$
```

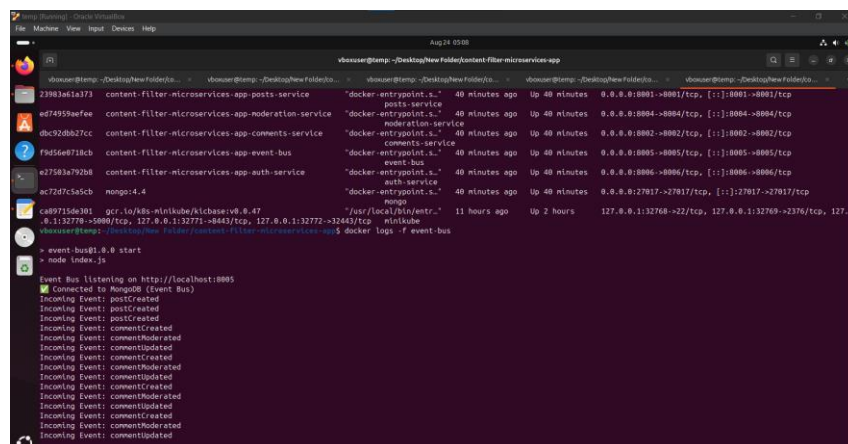
Application Working



Database

```
vboxuser@temp:~/Desktop/New Folder/content-filter-microservices-app$ docker exec -it mongo mongo
MongoDB shell version v4.4.29
connecting to: mongodb://127.0.0.1:27017/?compressors=disabled&gssapiServiceName=mongodb
Implicit session: session { "id" : UUID("d261d9bd-1e8d-4a78-8395-8ca7df3ae908") }
MongoDB server version: 4.4.29
Welcome to the MongoDB shell.
For interactive help, type 'help'.
For more comprehensive documentation, see
  https://docs.mongodb.com/
Questions? Try the MongoDB Developer Community Forums
  https://community.mongodb.com
...
The server generated these startup warnings when booting:
2025-08-24T04:22:52.826+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2025-08-24T04:22:53.721+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
...
> use microservices
switched to db microservices
> show collections
comments
events
moderationlogs
posts
queryposts
```

Communication Between Services



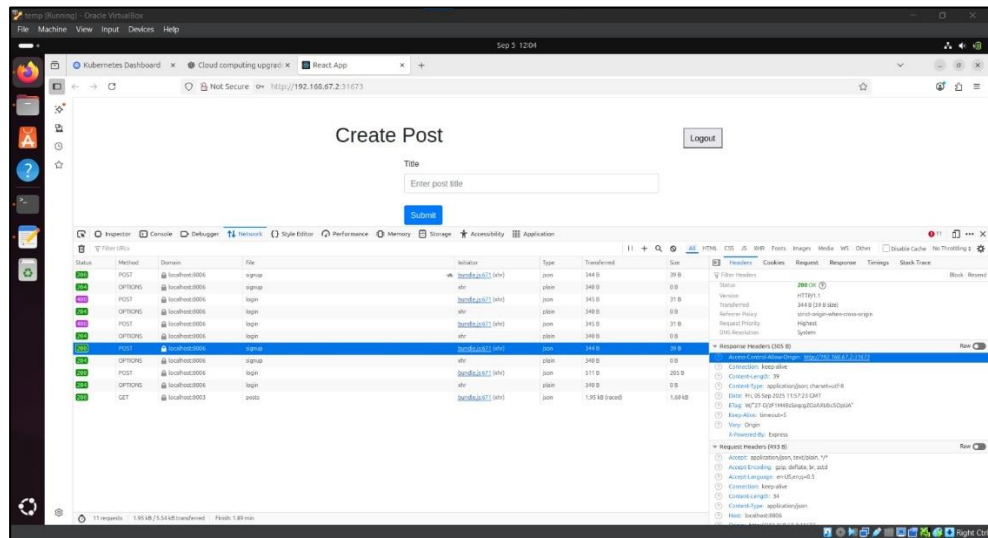
Scaling / High Availability

```
vboxuser@temp:~/Desktop/New Folder/content-filter-microservices-app$ kubectl scale deployment posts-service --replicas=3
deployment.apps/posts-service scaled
vboxuser@temp:~/Desktop/New Folder/content-filter-microservices-app$ kubectl get pods -w
```

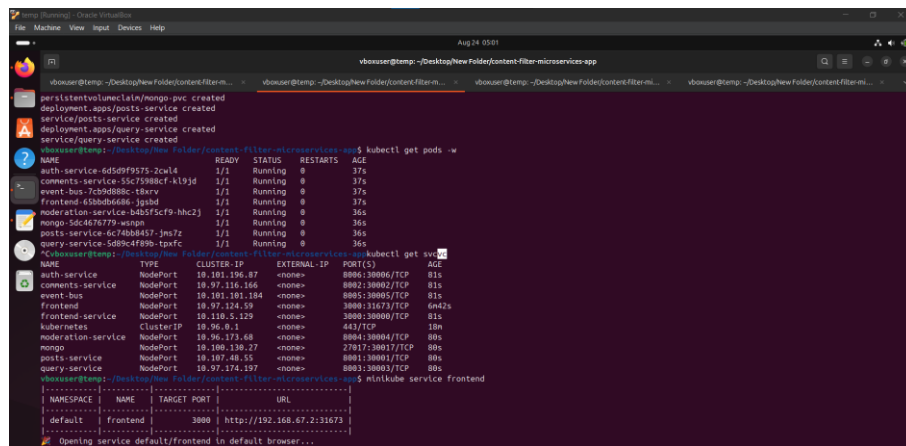
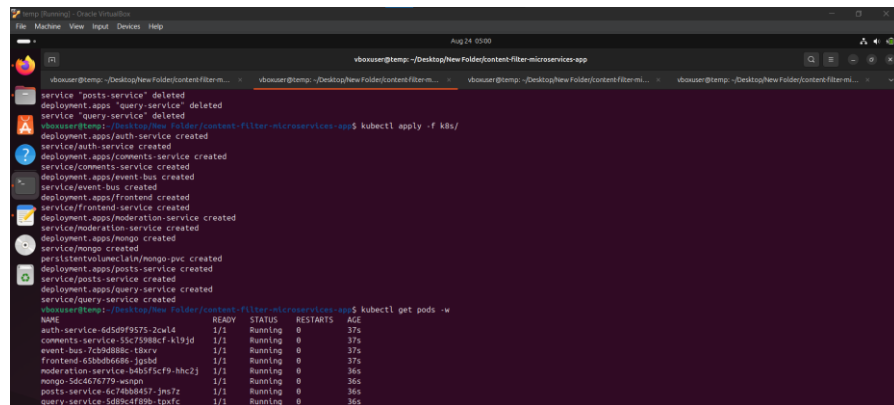
NAME	READY	STATUS	RESTARTS	AGE
auth-service-6d5d9f9575-2cw14	1/1	Running	0	27m
comments-service-55c75988cf-k19jd	1/1	Running	0	27m
event-bus-7cb9d888c-t8xrv	1/1	Running	0	27m
frontend-65bbdb6686-jgsbd	1/1	Running	0	27m
moderation-service-b4b5f5cf9-hhc2j	1/1	Running	0	27m
mongo-5dc4676779-wsnpn	1/1	Running	0	27m
posts-service-6c74bb8457-djdr	1/1	Running	0	24s
posts-service-6c74bb8457-jms7z	1/1	Running	0	27m
posts-service-6c74bb8457-s8cm5	1/1	Running	0	24s
query-service-5d89c4f89b-tpxfc	1/1	Running	0	27m

Security

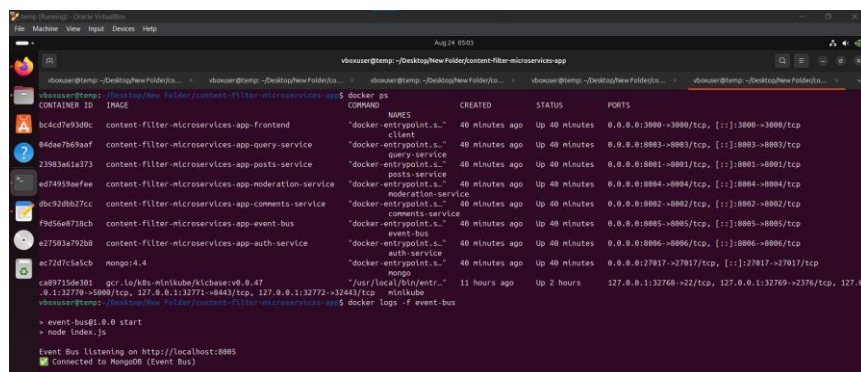
CORS is configured to allow requests only from the frontend origin (<http://192.168.67.2:31673>), ensuring secure access and preventing malicious requests from unauthorized origins.



Deployment Setup



Logs / Debugging



4. Challenges Faced

1. Environment & Virtualization Issues

- Trouble with enabling Intel VT-x virtualization in BIOS.
- Difficulty in recovering data from broken VM and attaching old .vdi disks.

2. Docker & Permissions

- Confusion about Docker image naming inside Minikube vs. YAML manifests.

3. Microservices & Networking

- Struggled with service-to-service communication inside Kubernetes.
- Confusion around MongoDB connection strings in Docker/K8s environments.

4. CI/CD Setup (Jenkins)

- Unsure how to trigger Postman collections automatically on new branch merges.
- Learning curve in integrating build, test, and deploy stages for microservices.

5. Data Persistence & Volume Management

- Unsure how to configure persistent storage for MongoDB in Docker/Kubernetes.
- Challenge in making data survive container restarts.

6. Resource Constraints & Configurations

- Limited VM resources (CPU, RAM, disk space) caused crashes.
- Confusion about YAML manifests (image names, ports, replicas).
- Struggled with configuring minikube load balancing / ingress for frontend access.

5. Lesson Learned

1. Virtualization & Backup Practices

- Always enable Intel VT-x before running VM-based projects.
- Keep snapshots of VMs before major changes to avoid data loss.
- Learned to attach and recover old .vdi disks to new VMs.

2. Docker Best Practices

- Must run Docker with proper permissions (docker group or sudo).
- Use consistent image names and tags across docker build and K8s YAMLs.

3. Kubernetes & Minikube

- Need to map correct image names in deployment YAMLs.
- Learned difference between ClusterIP, NodePort, and LoadBalancer.
- Understood importance of persistent volumes for databases.

4. CI/CD Automation

- Learned how to integrate Postman/Newman tests into Jenkins pipelines.
- Learned to trigger builds on branch merges with GitHub/GitLab webhooks.

6. General Cloud & DevOps Mindset

- Cloud-native projects require modular thinking.
- Small configs can break deployments.
- Importance of testing deployments early (local Docker → Minikube → CI/CD).